Análise e Redução do Volume de Tráfego do FACEIT (CS2): Captura Global, Classificação, Filtragem e Agregação em Fluxos

GRUPO 6

Ivo Costa Sousa – pg63976
João Afonso Almeida Sousa – pg62405
João Carlos Teixeira Neiva – pg63978

Novembro de 2026

Resumo

Este relatório, desenvolvido para a Unidade Curricular de Qualidade de Serviço na Internet, analisa o tráfego gerado pela plataforma FACEIT durante uma partida de CS2. A partir de uma captura global com o Wireshark (326 122 pacotes, 206,9 MB, cerca de 30 minutos), o serviço é classificado num plano de jogo (um único fluxo UDP, 79 % dos bytes, sem marcação DSCP e com um pacote a cada 15,6 ms) e num plano de controlo (DNS, TLS e QUIC, cerca de 2 % dos bytes). São estudados os critérios de filtragem mais eficazes e quantificada a redução de volume obtida por filtragem (21,3 %), truncagem de pacotes (87,5 %), *sampling* combinado com truncagem (98,9 %) e agregação em fluxos com o Argus e com o tshark (99,97 % e 99,96 %, respetivamente).

1 Parte 1: Objetivos e Metodologia

1.1 Objetivos

Este trabalho consolida as etapas estudadas sobre captura e análise de tráfego, tendo como serviço de estudo o FACEIT, plataforma competitiva de partidas de CS2 que exige a execução de um cliente de *Anti-Cheat* próprio [4]. Os objetivos definidos foram:

- **Captura global:** gerar tráfego real do serviço e capturar todo o tráfego da máquina com o Wireshark [1].
- **Classificação e filtragem:** estudar os critérios mais eficazes, tanto em filtros de captura como em filtros de visualização.
- **Redução de volume:** quantificar a percentagem de otimização face à captura global por filtragem, truncagem de pacotes e *sampling*.
- **Agregação em fluxos:** explorar ferramentas de agregação de pacotes em fluxos a partir de ficheiros pcap.

1.2 Captura

O Wireshark correu na própria máquina de jogo (`192.168.1.82`), ligada por cabo, sem filtro de captura e com pacotes inteiros, durante 1 818,6 s (cerca de 30 minutos) e com o FACEIT *Anti-Cheat* ativo. Antes da captura final realizou-se um ensaio num servidor normal de CS2, fora do FACEIT, para validar o procedimento.

1.3 Estratégia de Análise

Optou-se por uma única captura global, convertida para pcap simples (`editcap -F pcap`) para servir de *baseline*. Todos os cenários reduzidos foram derivados *offline* desse mesmo ficheiro, garantindo que a comparação

incide exatamente sobre o mesmo tráfego. Recorreu-se ao Wireshark/tshark, editcap, capinfos, tcpdump e ao Argus [3]. A percentagem de otimização foi calculada, em bytes (tamanho do ficheiro) e em número de pacotes, por:

$$\text{redução} = \left(1 - \frac{\text{tamanho reduzido}}{\text{tamanho global}}\right) \times 100.$$

O tamanho do ficheiro inclui 16 B de cabeçalho por pacote no formato pcap. O campo *Data size* do capinfos não reflete a truncagem, pelo que não foi utilizado.

2 Parte 2: Caracterização do Tráfego

2.1 Visão Geral

A captura global contém 326 122 pacotes e 201,6 MB de dados (212,6 MB em pcapng e 206,9 MB em pcap), com uma média de 618 B por pacote e 179 pacotes/s. A composição por classe encontra-se na Tabela 1; as classes sobrepõem-se parcialmente, pelo que não devem ser somadas.

Tabela 1: Composição do tráfego global.

Classe	Pacotes	Bytes	% Pacotes	% Bytes
Servidor de jogo (`141.95.32.214`)	202 086	159 679 469	62,0	79,2
UDP :19336 (`104.29.146.69`)	85 785	17 938 347	26,3	8,9
TCP (todo)	19 394	12 376 258	5,9	6,1
QUIC (todo)	7 226	9 622 529	2,2	4,8
mDNS / SSDP / DNS	4 576	1 307 245	1,4	0,6
Controlo FACEIT (`193.41.200.101/102`)	3 995	3 338 873	1,2	1,7

A primeira linha conta todos os pacotes com o IP do servidor (202 086), incluindo 80 que não pertencem à conversa UDP principal (202 006 pacotes, o valor usado no resto do relatório). Estes 80 pacotes são ICMP (Figura 1) e correspondem a um *traceroute* (Secção 2.5).

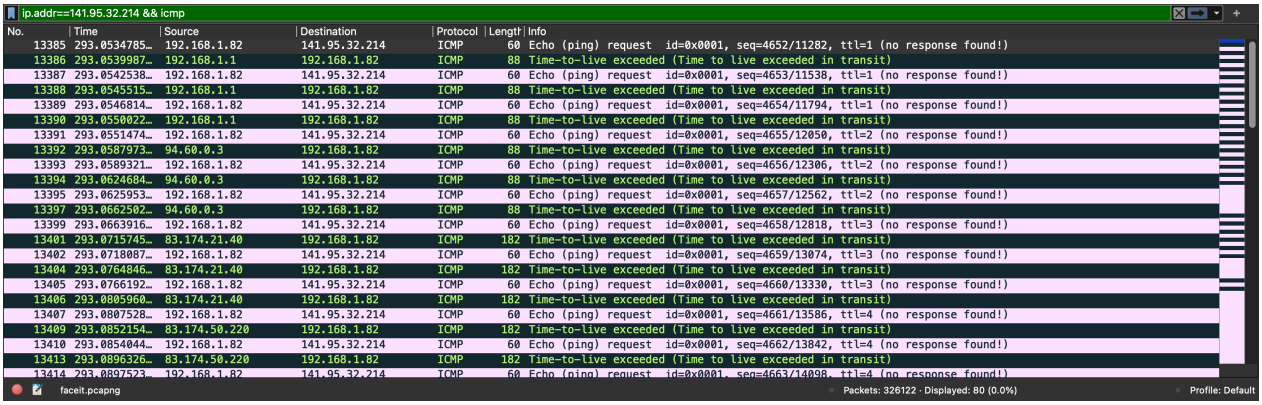


Figura 1: Filtro `ip.addr==141.95.32.214 && icmp`: os 80 pacotes ICMP com o IP do servidor, ou seja, sondas com TTL crescente e respostas *Time-to-live exceeded* dos *routers* intermédios.

2.2 Plano de Jogo

A conversa UDP `192.168.1.82:50088` ↔ `141.95.32.214:27031` domina a captura (Figura 2). Inicia-se aos 293,8 s, dura 1 520,5 s e totaliza 202 006 pacotes e 159,7 MB. Os seus principais indicadores são:

- **Cliente → servidor:** 96 786 pacotes (70,7 MB), com média de 730 B por pacote.
- **Servidor → cliente:** 105 220 pacotes (89,0 MB), com média de 846 B por pacote.
- **Débito:** praticamente constante, com cerca de 133 pacotes/s e 0,84 Mbit/s (aproximadamente 40 000 pacotes por cada 300 s).
- **Tamanho dos pacotes:** concentrado entre 640 e 1 279 B (69,3 % dos pacotes), com média de 790 B.

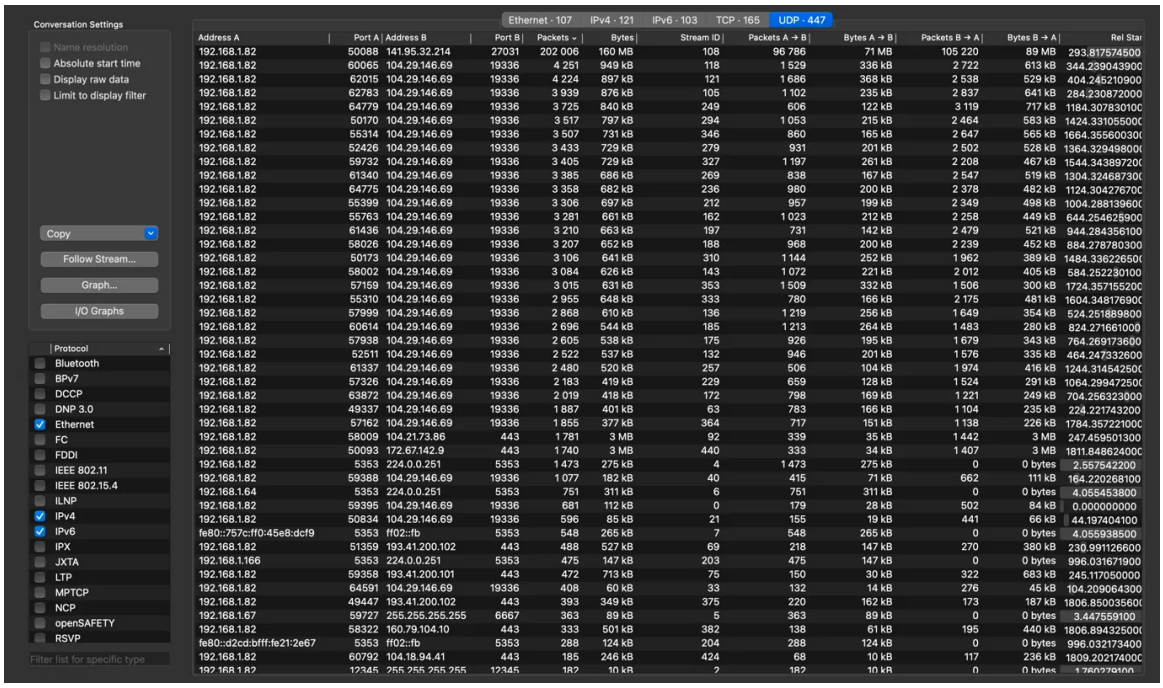


Figura 2: *Conversations* → UDP, ordenado por pacotes: o fluxo para `141.95.32.214:27031` domina.

2.3 Características de QoS do Fluxo de Jogo

Como a Unidade Curricular se centra na Qualidade de Serviço, caracterizou-se o fluxo de jogo do ponto de vista da marcação e da regularidade temporal, medidas no ponto de captura (a máquina de jogo). Os 202 006 pacotes têm DSCP 0 nos dois sentidos, ou seja, o tráfego é tratado como *best effort*, sem qualquer marcação de QoS. A Tabela 2 resume o tempo entre pacotes consecutivos de cada sentido. Neste relatório, o *jitter* é medido pelo desvio-padrão desse tempo (7,58 ms no sentido servidor → cliente e 4,02 ms no sentido inverso).

Tabela 2: Tempo entre pacotes consecutivos do fluxo de jogo (ms).

Sentido	Pacotes	Média	Mediana	p95	p99	Máximo	Desvio-padrão
Servidor → cliente	105 220	14,45	15,60	16,05	16,52	1 000,4	7,58
Cliente → servidor	96 786	15,71	15,64	16,65	17,11	518,6	4,02

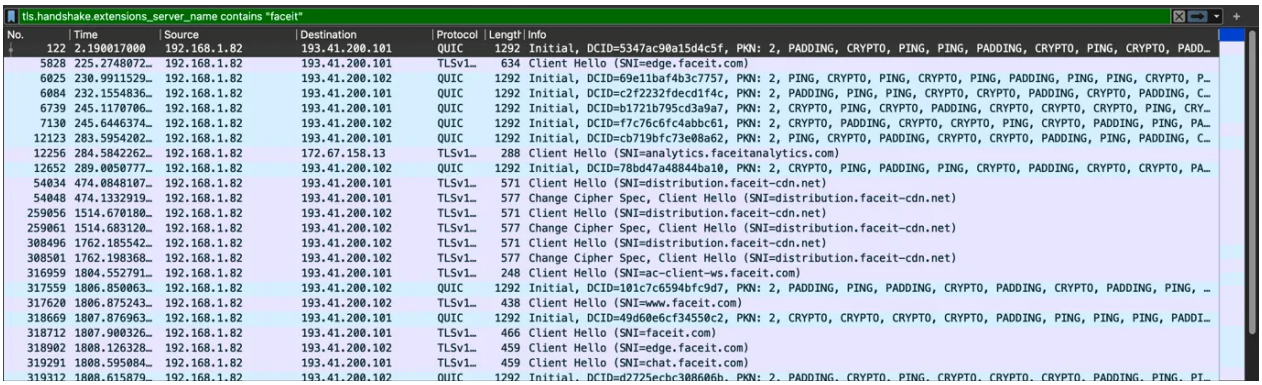
As medianas de cerca de 15,6 ms correspondem a aproximadamente 64 pacotes/s por sentido, e os percentis 95 e 99 ficam muito próximos da mediana, o que mostra um fluxo muito regular. No sentido servidor → cliente, a média abaixo da mediana e o desvio-padrão maior sugerem pacotes que chegam em rajadas, e os máximos de 1 000 ms e 519 ms correspondem a interrupções isoladas. Trata-se, portanto, de um fluxo de baixo débito mas sensível à latência e ao *jitter*.

2.4 Plano de Controlo

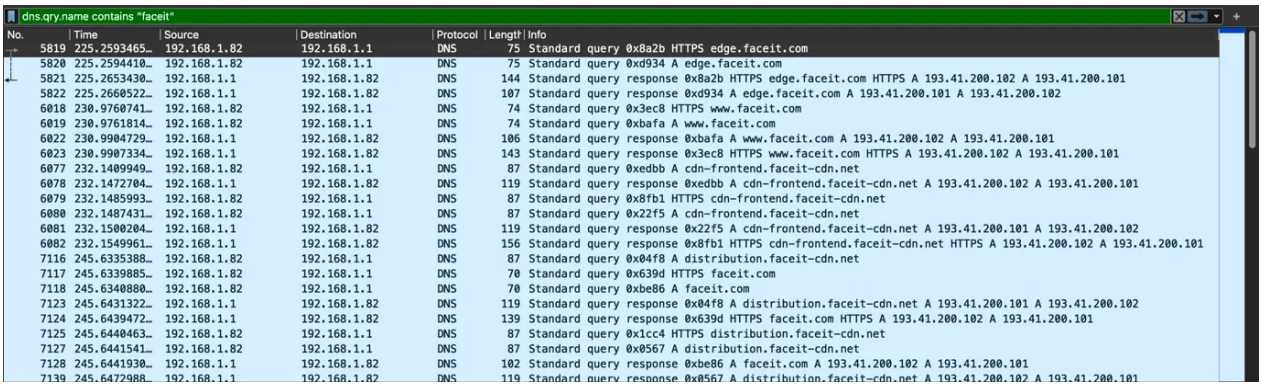
O DNS e o SNI (em TLS e QUIC) permitem identificar inequivocamente o serviço (Figura 3). Foram observados os nomes `www.faceit.com`, `edge.faceit.com`, `chat.faceit.com`, `faceit.com`, `cdn-frontend.faceit-cdn.net`, `distribution.faceit-cdn.net` e `ac-client-ws.faceit.com`, todos resolvidos para `193.41.200.101` e `193.41.200.102`, a que se junta `analytics.faceitanalytics.com` (Cloudflare). O primeiro contacto ocorre aos 2,2 s (QUIC para `www.faceit.com`), surge uma nova rajada de pedidos entre os 225 e os 245 s, antes de o fluxo de jogo começar, e um último contacto a `ac-client-ws.faceit.com` aos 1 804,6 s. Este plano é reduzido: 3 995 pacotes e 3,3 MB (1,7 % dos bytes).

2.5 Tráfego Auxiliar e Ruído

Traceroute. Aos 293,05 s, pouco antes de o fluxo UDP começar, o cliente envia 51 *Echo requests* ICMP para o servidor, com TTL de 1 a 17 (três sondas por TTL) e o texto `icmp-trace-packet` no *payload*, ou



(a) `tls.handshake.extensions_server_name contains "faceit"` (TLS e QUIC).



(b) `dns.qry.name contains "faceit"`.

Figura 3: Identificação do plano de controlo por filtros de visualização.

seja, um *traceroute* (Figura 1, na Secção 2.1). Recebe 26 mensagens *Time-to-live exceeded* de nove *routers* intermédios (os saltos 7, 8 e 11 não respondem) e 3 *Echo replies* do próprio servidor, no salto 17. Os 51 + 26 + 3 = 80 pacotes são exatamente os que distinguem os 202 086 da Tabela 1 dos 202 006 da conversa UDP. Não foi confirmado qual o processo que o gera (possivelmente o cliente do jogo).

As respostas permitem estimar o RTT até ao servidor, *a priori* um valor indicativo, já que os *routers* geram ICMP com baixa prioridade: cerca de 0,4 ms no primeiro salto (*router* doméstico), 3,6 ms no segundo, 4–5 ms nos saltos 3 e 4, 29 ms no salto 5 e cerca de 55,7 ms até ao servidor (média das três *Echo replies*).

Ruído. Os fluxos UDP para `104.29.146.69:19336` (um novo fluxo por minuto, 85 785 pacotes) são independentes do jogo, uma vez que surgiam também na captura de ensaio; a sua origem não foi identificada. Há ainda uma ligação TCP de longa duração (1 818 s) para `185.25.182.20:27021`, provavelmente do cliente Steam, e tráfego da rede local (mDNS, SSDP, *broadcasts*) e outras ligações HTTPS/QUIC.

3 Parte 3: Critérios de Classificação e Filtragem

3.1 Filtros Estudados

O filtro de captura (BPF) [2] apenas tem acesso aos cabeçalhos (IP, protocolo, portas), pelo que critérios baseados em nomes (DNS/SNI) só são possíveis como filtros de visualização. A eficácia de cada critério foi avaliada através da **cobertura** (pacotes do jogo apanhados / pacotes do jogo) e da **precisão** (pacotes do jogo apanhados / pacotes apanhados), tomando como referência a conversa do jogo (202 006 pacotes). Os resultados constam da Tabela 3.

3.2 Discussão

Todos os filtros baseados em IP ou em portas atingem uma cobertura de 100 % do fluxo de jogo. A gama de portas apanha apenas 34 pacotes a mais do que o filtro por IP (precisão de 99,98 %) e tem a vantagem de não depender do IP do servidor, que muda de partida para partida. Em contrapartida, o filtro apenas por protocolo (`udp`) revela-se demasiado abrangente, já que 33 % do que apanha não pertence ao jogo.

Tabela 3: Critérios de classificação e respetiva eficácia.

Critério	Filtro de captura (BPF)	Filtro de visualização	Pacotes	Precisão
Servidor (IP)	`udp and host 141.95.32.214`	`ip.addr==141.95.32.214 && udp`	202 006	100 %
Qualquer protocolo com o IP	`host 141.95.32.214`	`ip.addr==141.95.32.214`	202 060	99,97 %
Gama de portas	`udp portrange 27000-27050`	`udp.port>=27000 && udp.port<=27050`	202 040	99,98 %
Só UDP	`udp`	`udp`	300 571	67,2 %
Controlo (IP)	`host 193.41.200.101 or host 193.41.200.102`	`ip.addr==193.41.200.101 || ip.addr==193.41.200.102`	3 995	—
Controlo (nome)	(não aplicável)	`dns.qry.name contains "faceit"; tls.handshake.extensions_server_name contains "faceit"`	—	—
Jogo + controlo (IPs)	`host servidor or host controlo (2 IPs)`	`--`	206 055	—
Portas + controlo	`udp portrange 27000-27050 or host 193.41.200.101 or host 193.41.200.102`	`--`	206 035	—

O filtro BPF `host 141.95.32.214` apanha 202 060 pacotes, ou seja, mais 54 do que a conversa UDP: são as 51 sondas e as 3 respostas do *traceroute*. O filtro de visualização `ip.addr==141.95.32.214` chega aos 202 086 da Tabela 1 porque o Wireshark também compara o IP do pacote original embebido nas 26 mensagens *Time-to-live exceeded*, que o BPF (que só olha para o cabeçalho exterior) não vê. Os filtros que incluem `udp` deixam este *traceroute* de fora, o que é aceitável para o estudo do jogo.

Conclui-se que a deteção do fluxo de jogo é mais eficaz através da combinação de três critérios: transporte UDP, gama de portas 27xxx e dominância por volume (*Conversations* ordenado por bytes), sendo o serviço confirmado pelo DNS/SNI do plano de controlo. Para uma captura ao vivo, em que o IP do servidor ainda não é conhecido, o filtro mais adequado é o último da Tabela 3, que junta a gama de portas aos dois IPs do plano de controlo.

4 Parte 4: Redução de Volume

4.1 Resultados

A Tabela 4 e a Figura 4 apresentam os cenários avaliados. Todas as percentagens são calculadas face ao *baseline* (A) de 326 122 pacotes e 206 864 624 B.

4.2 Filtragem

Filtrar o fluxo de jogo reduz apenas 21,25 % dos bytes, mas 38,06 % dos pacotes. Esta assimetria explica-se pelo facto de o jogo já representar 79 % dos bytes capturados, enquanto o ruído é composto por muitos pacotes pequenos: os 85 785 pacotes de `:19336` correspondem a 26 % dos pacotes, mas a apenas 9 % dos bytes. Incluir o plano de controlo no filtro tem um custo reduzido (19,58 % a 19,60 % contra 21,25 %) e descreve o serviço de forma mais completa.

4.3 Truncagem de Pacotes (Snaplen)

Não capturar o pacote inteiro é a técnica com maior impacto individual. Os cabeçalhos Ethernet, IPv4 e UDP ocupam 42 B, pelo que, com *snaplen* 96, se preservam os cabeçalhos e mais 54 B de *payload*. Com pacotes de 790 B em média, a redução total sobe de 21,25 % para 87,50 % (84,38 % com 128 B e 90,63 % com 64 B). O custo é a perda do restante *payload*.

4.4 Sampling

Guardar 1 pacote em cada 10 (por ordem de pacote) proporciona uma redução de 92,08 % sem truncagem e de 98,91 % com *snaplen* 96 (2,26 MB, dos quais cerca de 0,32 MB correspondem a cabeçalhos de pacote

Tabela 4: Redução de volume face à captura global.

	Cenário	Pacotes	Bytes	Red. bytes	Red. pacotes
A	Captura global (*baseline*)	326 122	206 864 624	0 %	0 %
B1	Filtro `udp and host` servidor	202 006	162 905 309	21,25 %	38,06 %
B3	Filtro gama de portas 27000–27050	202 040	162 951 481	21,23 %	38,05 %
B4	Filtro só `udp`	300 571	193 613 964	6,41 %	7,83 %
B5	Filtro jogo + controlo FACEIT (IPs)	206 055	166 312 206	19,60 %	36,82 %
B6	Filtro portas + controlo FACEIT	206 035	166 354 274	19,58 %	36,82 %
C1	B1 + *snaplen* 128	202 006	32 311 732	84,38 %	38,06 %
C2	B1 + *snaplen* 96	202 006	25 856 584	87,50 %	38,06 %
C3	B1 + *snaplen* 64	202 006	19 392 644	90,63 %	38,06 %
C4	B5 + *snaplen* 96	206 055	26 333 772	87,27 %	36,82 %
C5	B6 + *snaplen* 96 (filtro recomendado)	206 035	26 333 156	87,27 %	36,82 %
D1	B1 + *sampling* 1/10	20 200	16 376 628	92,08 %	93,81 %
D2	B1 + *snaplen* 96 + *sampling* 1/10	20 200	2 262 424	98,91 %	93,81 %
E1	Fluxos Argus, global (registos de 5 s)	—	1 004 872	99,51 %	—
E2	Fluxos Argus, global (1 registo/fluxo, CSV)	—	64 839	99,97 %	—
E3	Fluxos Argus, só servidor (CSV, 1 fluxo)	—	229	≈100 %	—
E4	Conversas tshark, global (texto, UDP+TCP)	—	89 824	99,96 %	—

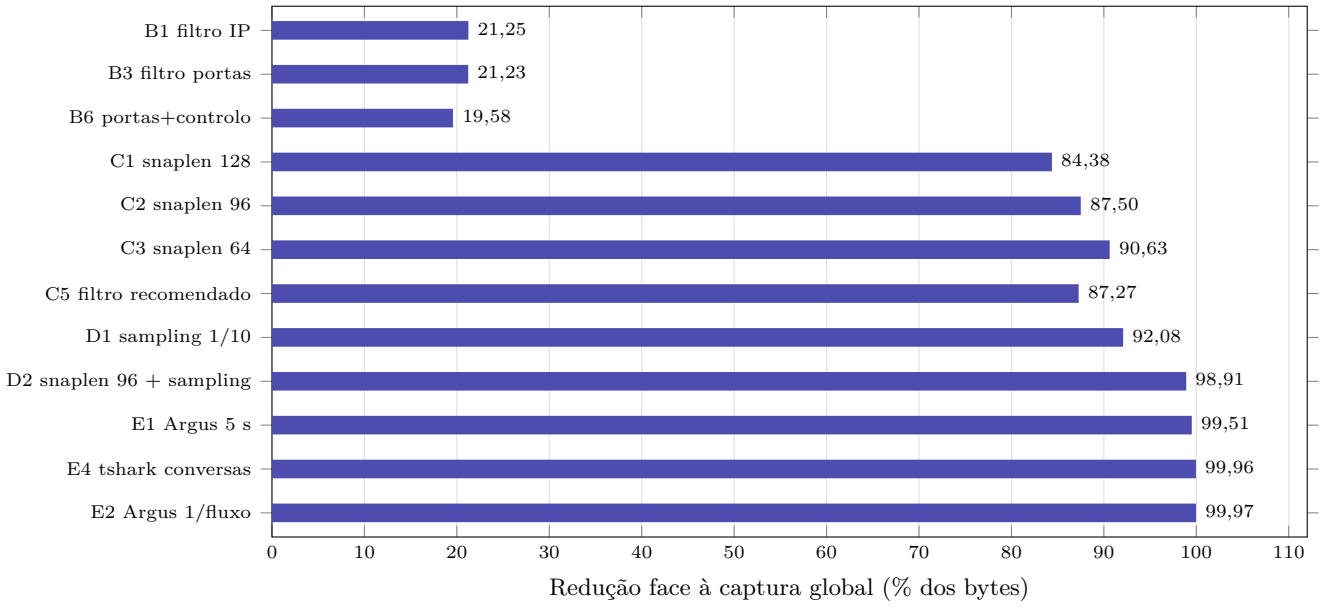


Figura 4: Redução de bytes por cenário.

do formato pcap). As estatísticas de volume mantêm-se fiáveis: extrapolando $\times 10$, o erro é de $-0{,}003\,\%$ nos pacotes e de $+0{,}54\,\%$ nos bytes. Perdem-se, contudo, as sequências de pacotes consecutivos.

4.5 Impacto nas Métricas de QoS

As técnicas não são equivalentes para quem quer medir QoS. A filtragem e o *snaplen* 96 mantêm todos os pacotes, as marcas temporais e o cabeçalho IP (onde está o DSCP), pelo que ainda permitem calcular o tempo entre pacotes e o *jitter* da Tabela 2. O *sampling* 1/10 preserva volumes e distribuições de tamanhos, mas os intervalos medidos entre pacotes amostrados deixam de corresponder aos reais. A agregação em fluxos conserva o débito médio e a duração, mas perde o *jitter*.

5 Parte 5: Agregação de Pacotes em Fluxos

5.1 Ferramentas e Processo

Exploraram-se duas ferramentas sobre o mesmo `A_global.pcap`. O Argus (`argus -r f.pcap -w f.argus`) emite registos de estado de cerca de 5 s por fluxo, e o `racluster` agrupa-os num único registo por fluxo (5-tuplo), exportado para CSV através do `ra`. O tshark produz, com `-z conv,udp -z conv,tcp`, uma lista de conversas UDP e TCP em texto.

5.2 Resultados

A Tabela 5 compara as saídas obtidas a partir do *baseline* de 206,9 MB.

Tabela 5: Agregação em fluxos: Argus e tshark.

Saída	Registos	Tamanho (B)	Redução	Protocolos
Argus, registos de 5 s (binário)	—	1 004 872	99,51 %	todos
Argus, 1 registo por fluxo (binário)	797	122 656	99,94 %	todos
Argus, 1 registo por fluxo (CSV)	797	64 839	99,97 %	todos
tshark conversas (texto)	612	89 824	99,96 %	UDP e TCP

As duas ferramentas chegam a ordens de grandeza semelhantes (cerca de 65 kB a 90 kB, mais de 99,9 %), mas diferem no que registam:

- **Cobertura de protocolos:** o tshark só lista UDP e TCP (447 e 165 conversas, tal como mostram os separadores das *Conversations*). O Argus regista também ICMPv6, ARP, IGMP, ICMP e LLDP, daí os 797 fluxos (mais um registo de gestão do Argus), dos quais cerca de 200 não são UDP nem TCP.
- **Fluxos TCP:** o Argus tem 149 fluxos TCP e o tshark 165. Na captura existem 149 5-tuplos TCP distintos, que é o que o `racluster` agrega. A diferença de 16 conversas vem de quatro ligações a `192.168.1.166:8009` (porta tipicamente usada por dispositivos Google Cast na rede local) que repetem o mesmo 5-tuplo cinco vezes: o tshark conta cada nova tentativa de ligação como uma conversa distinta, o Argus junta-as num único registo ($4 \times 4 = 16$). Isto significa que o Argus agrega por 5-tuplo e o tshark por ligação.
- **Precisão dos totais:** o tshark em texto arredonda os bytes (por exemplo, 159 MB para o fluxo de jogo), enquanto o Argus conserva os valores exatos.

Os totais do fluxo de jogo no Argus (70 661 525 B do cliente e 89 011 664 B do servidor) somam 159 673 189 B, exatamente o volume de dados do pcap filtrado. Aplicado ao filtro do servidor, o fluxo de jogo fica reduzido a um único registo de 229 B no CSV. A agregação conserva o 5-tuplo, o instante de início, a duração e os pacotes e bytes por sentido, mas perde a temporização e o tamanho de cada pacote, bem como o *payload*. Os registos de 5 s do Argus conservam ainda a evolução temporal do fluxo.

6 Parte 6: Conclusões

6.1 Síntese

- **Estrutura do serviço:** plano de jogo (UDP `141.95.32.214:27031`, 79 % dos bytes) e plano de controlo (DNS/TLS/QUIC para `193.41.200.101/102`, 1,7 % dos bytes).
- **Classificação:** os critérios mais eficazes para detetar o fluxo de jogo são o transporte UDP, a gama de portas 27000–27050 e a dominância por volume; o DNS/SNI confirma o serviço. O filtro apenas por protocolo é demasiado abrangente.
- **QoS do fluxo de jogo:** DSCP 0 em todos os pacotes, um pacote a cada 15,6 ms em cada sentido com pouco desvio, e um RTT até ao servidor de cerca de 56 ms.
- **Redução de volume:** a filtragem poupa pouco em bytes (21 %), dado que o jogo domina a captura. As maiores poupanças resultam do *snaplen* (87,5 %), do *sampling* (98,9 % em conjunto) e da agregação em fluxos (99,5 % a 99,97 %).
- **Escolha da técnica:** depende da utilização pretendida: cabeçalhos e tempos por pacote ⇒ filtro e *snaplen* 96; estatísticas de volume ⇒ *sampling*; contabilização e retenção prolongada ⇒ fluxos.

6.2 Configuração de Captura Recomendada

Para uma captura ao vivo deste serviço recomenda-se o filtro de portas e IPs de controlo com *snaplen* 96, por exemplo com o `dumpcap`:

```
dumpcap -i <n> -f "udp portrange 27000-27050 or host 193.41.200.101 or host 193.41.200.102" -s 96 -w captura.pcapng
```

Aplicada *offline* à captura global, esta configuração (C5) guarda 206 035 pacotes em 26,3 MB, uma redução de 87,27 % dos bytes, e mantém os cabeçalhos, as marcas temporais e as *queries* DNS (70 a 87 B, que cabem inteiras). Em contrapartida, as respostas DNS (até 156 B) ficam truncadas e o SNI deixa de ser legível: no TLS surge para lá dos 96 B do pacote e, nos QUIC *Initial* (1 292 B), só se obtém decifrando o pacote completo. Com este *snaplen*, o plano de controlo é portanto identificado pelos IPs e não pelo nome do serviço. Esta configuração não foi validada numa captura ao vivo.

6.3 Limitações

Foi analisada uma única captura de uma única partida, e o IP e a porta do servidor mudam entre partidas, pelo que a gama de portas 27000–27050 pode não se aplicar a outros servidores. Os cenários foram derivados *offline* da captura global, e não capturados ao vivo. O *sampling* aplicado é determinístico (1 em 10 por ordem de pacote) e as percentagens em ficheiro incluem o cabeçalho de 16 B por pacote do formato pcap. O tempo entre pacotes e o RTT foram medidos no ponto de captura e incluem, por isso, o efeito da rede e do próprio sistema de captura. A agregação em fluxos usou apenas o Argus e o tshark.

Referências

[1] Wireshark Foundation, *Wireshark User's Guide*. https://www.wireshark.org/docs/wsug_html_chunked/

[2] The Tcpdump Group, *pcap-filter(7) — packet filter syntax*. https://www.tcpdump.org/manpages/pcap-filter.7.html

[3] QoSient, *Argus — Auditing Network Activity*. https://openargus.org

[4] FACEIT, *What is FACEIT Anti-cheat and how does it work?* https://support.faceit.com/hc/en-us/articles/9394666828188-What-is-FACEIT-Anti-cheat-and-how-does-it-work

A Comandos Utilizados

```
# Baseline
capinfos faceit.pcapng
editcap -F pcap faceit.pcapng A_global.pcap
```

```
# Caracterizacao
tshark -r A_global.pcap -q -z io,phs
tshark -r A_global.pcap -q -z conv,udp
tshark -r A_global.pcap -q -z plen,tree,"ip.addr==141.95.32.214"
tshark -r A_global.pcap -Y 'dns.flags.response==1 && dns.qry.name contains "faceit"' -T fields -e frame.time_relative -e dns.
     qry.name -e dns.a
tshark -r A_global.pcap -Y 'tls.handshake.extensions_server_name contains "faceit"' -T fields -e frame.time_relative -e ip.
     dst -e tls.handshake.extensions_server_name

# Traceroute (80 pacotes ICMP com o IP do servidor fora da conversa UDP)
tshark -r A_global.pcap -Y "ip.addr==141.95.32.214 && icmp" -q -z io,phs
tshark -r A_global.pcap -Y "icmp && ip.addr==141.95.32.214" -T fields -e frame.time_relative -e ip.src -e ip.ttl -e icmp.type
      -e icmp.seq_le
# RTT por salto: emparelhamento das sondas com as respostas pelo numero de sequencia (Python)

# QoS do fluxo de jogo (DSCP e instantes; os intervalos foram calculados em Python)
tshark -r B1_udp_host.pcap -T fields -e ip.dsfield.dscp | sort | uniq -c
tshark -r B1_udp_host.pcap -Y "ip.src==141.95.32.214" -T fields -e frame.time_epoch

# Filtros (BPF, aplicados offline)
tcpdump -r A_global.pcap -w B1_udp_host.pcap "udp and host 141.95.32.214"
tcpdump -r A_global.pcap -w B3_portrange.pcap "udp portrange 27000-27050"
tcpdump -r A_global.pcap -w B4_udp.pcap "udp"
tcpdump -r A_global.pcap -w B5_game_ctrl.pcap "host 141.95.32.214 or host 193.41.200.101 or host 193.41.200.102"
tcpdump -r A_global.pcap -w B6_portrange_ctrl.pcap "udp portrange 27000-27050 or host 193.41.200.101 or host 193.41.200.102"

# Snaplen e sampling
editcap -s 96 B1_udp_host.pcap C2_snaplen96.pcap
editcap -s 96 B6_portrange_ctrl.pcap C5_portrange_ctrl_snaplen96.pcap
tshark -r C2_snaplen96.pcap -Y "frame.number % 10 == 0" -F pcap -w D2_snaplen_sampling.pcap

# Fluxos (Argus)
argus -r A_global.pcap -w E1_global.argus
racluster -r E1_global.argus -m proto saddr sport daddr dport -w E2_global_cluster.argus
ra -r E2_global_cluster.argus -n -c , -s stime dur proto saddr sport daddr dport spkts dpkts sbytes dbytes > E2_global_flows.
     csv

# Fluxos (tshark) e medicao
tshark -r A_global.pcap -q -z conv,udp -z conv,tcp > flows_tshark.txt
capinfos -T -m -M -c -s *.pcap
```